

USBSID-Pico Midi manual

Author: LouD - generated on 2026-09-02

Table of Contents

Disclaimer	3
Quick start	3
Notes, velocity and aftertouch	3
Channel routing	4
Control Change reference	4
Voice waveform and envelope	4
Filter and volume	5
SID / voice selection and channel mode	6
Modulation and timing	6
Second LFO	7
Unison	7
Panic and reset	8
Cynthcart	8
Deprecated	9
Not wired	9
Patches	9
Factory patches	9
Authoring your own patches over SysEx	10
Arp tables	11
Using a table	11
Authoring tables over SysEx	11
Saving your settings	12
FMOp1	12
Targeting a channel at FMOp1	13
FMOp1 patches	13
Live control	14
Known limitations	14
A note on tuning	15
Development	15
License	15
Software License ~ GNUv2	15
Configtool Software License ~ EPLv2	15
USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0	15
USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0	16

USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0	16
USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0	16

Disclaimer

I do this stuff in my free time for my enjoyment. Since I like to share my joy in creating this with everyone I try my best to provide a working PCB and Firmware. I am in no way an electronics engineer and can give **no guarantees** that this stuff does not break or damage your hardware, computer, phone, or whatever you try to hook it up to. Be sure to take great care when inserting any real MOS SID chips into the board. While everything has been tested with real chips, this is in no way a guarantee that nothing could go wrong. Use of this board and firmware at your own risk! I am in no way responsible for your damaged hardware. That being said, have fun!

Quick start

USBSID-Pico shows up as a class-compliant USB-MIDI device - no driver install needed on any platform. Point any MIDI application, DAW, or live-coding environment (Strudel, VMPK, a hardware keyboard through a MIDI interface, ...) at it and start playing.

Out of the box, without touching any configuration:

- **Channel 1** uses every voice on every configured SID, fully polyphonic (up to 12 voices on a 4-SID board). Plug in a keyboard and play.
- **Channels 2-5** each get one SID chip exclusively (channel 2 → SID 1, channel 3 → SID 2, and so on), 3-voice polyphonic within that chip. This gives a sane multi-timbral split (one instrument per channel, one channel per chip) without touching a config tool.
- **Channels 6-16** are disabled (no voice mask) until you assign them yourself.
- Every channel defaults to a plain triangle wave with full sustain, so a note is immediately audible - no need to send a waveform or envelope CC first to hear anything.

If your board has an FMOpI-capable chip installed, see [FMOpI](#) below - it is a separate instrument with its own note path and its own smaller set of controls, not just another SID.

Notes, velocity and aftertouch

Standard MIDI Note On / Note Off (**0x90/0x80**) trigger and release voices. A Note On with velocity 0 is treated as a Note Off, matching how most keyboards send note releases under running status.

Velocity does nothing to the sound by default - this is deliberate, so a controller's velocity curve cannot accidentally stomp on the channel volume CC. Turn **CC_VELM** on (see the table below) to make velocity scale the decay stage instead (higher velocity = punchier, shorter decay).

Channel Aftertouch (**0xD0**) and Polyphonic Key Pressure (**0xA0**, applied channel-wide rather than per note) both drive whichever target **at_target** currently selects for that channel - filter cutoff by default, or channel volume, or LFO depth (a ready-made vibrato-on-pressure setup) if a loaded patch sets it differently.

Pitch Bend (**0xE0**) bends every voice currently held on the channel, live, using that channel's bend range in semitones (2 by default, or whatever the currently-loaded patch set it to - there is no

separate CC for bend range).

Channel routing

Every MIDI channel (1-16) has its own independent routing, entirely separate from which patch is loaded on it:

- **Voice mask** - which of the pool's voice slots (up to 12 on a 4-SID board, 3 per chip) this channel is allowed to use. Set up sensibly out of the box (see [Quick start](#)); reconfigured through the USBSID-Pico config tool, not through MIDI CCs.
- **SID / voice overrides** - `CC_SID1-CC_SID4` and `CC_VCE1-CC_VCE3` temporarily narrow a channel's own voice mask to one specific chip or one specific voice position, without changing the configured mask itself. Send the CC with any nonzero value to select, `0` to clear the override and go back to the full configured mask. Handy for quickly confining a part to one voice while leaving the channel's normal setup untouched for the next patch.
- **Polyphony** - `CC_MONO/CC_POLY` switch a channel between monophonic and fully polyphonic (within whatever it's currently allowed to use), silencing that channel's held notes on the switch. `CC_SPLY` is a lighter version of the same idea that does not cut off notes already playing.
- **Steal behaviour** - when a channel is out of free voices, the oldest-held note is stolen by default. Not exposed as a CC; this is a config-tool setting.

Control Change reference

All CC numbers below are this firmware's **compiled-in defaults** ([repo/src/midi_defs.h](#), `MIDI_DEFAULT_CCVALUES_INIT`). The whole map is itself reconfigurable (persisted alongside everything else in the MIDI flash blob, see [Saving your settings](#)), so if your board's CC map has been changed, these numbers will not match - check your own configuration first if a CC below doesn't seem to do anything.

Voice waveform and envelope

Each channel keeps one live "template" (waveform, envelope, pulse width) that gets stamped onto every new note and pushed live to every note the channel is currently holding - so a waveform or envelope CC change is heard immediately on a held chord, not just on the next note.

CC	Name	What it does
20	<code>CC_NOIS</code>	Toggle the noise waveform bit
21	<code>CC_PULS</code>	Toggle the pulse waveform bit
22	<code>CC_SAWT</code>	Toggle the sawtooth waveform bit
23	<code>CC_TRIA</code>	Toggle the triangle waveform bit
24	<code>CC_TEST</code>	Toggle the test bit
25	<code>CC_RMOD</code>	Toggle ring modulation

CC	Name	What it does
26	CC_SYNC	Toggle hard sync
27	CC_GATE	Manual gate open/close on every voice this channel currently holds - separate from the automatic gate a note-on/note-off already drives, for holding a drone open by hand
17	CC_ATT	Attack
18	CC_DEC	Decay
19	CC_SUS	Sustain
29	CC_REL	Release
1	CC_PWM	Pulse width (the modulation wheel on most controllers) - only audible when the pulse waveform is active
28	CC_NOTE	One-shot manual pitch entry for whatever this channel is currently holding

Waveform bits are combinable exactly like real SID hardware - turning on more than one at once gives the same thick, slightly metallic combined-waveform sound a real chip produces, not an error.

Filter and volume

Filter and volume are chip-wide registers, not per-voice - a CC here affects every SID chip currently in the channel's mask at once, immediately, regardless of how many notes are held.

CC	Name	What it does
32	CC_FFC	Filter cutoff frequency
33	CC_RES	Filter resonance
34	CC_FLT1	Route voice 1 through the filter
35	CC_FLT2	Route voice 2 through the filter
36	CC_FLT3	Route voice 3 through the filter
37	CC_FLTE	Route the external input through the filter
48	CC_3OFF	Disconnect voice 3 from the output entirely
49	CC_HPF	Enable the high-pass filter mode
50	CC_BPF	Enable the band-pass filter mode
51	CC_LPF	Enable the low-pass filter mode
7	CC_VOL	Channel volume

IMPORTANT

On real SID hardware, a voice routed into the filter (CC_FLT1-3/CC_FLTE) produces **no output at all** unless a filter mode (CC_HPF/CC_BPF/CC_LPF) is also enabled. This is not a bug to work around - it is how the chip actually

behaves. If a sound goes silent the moment you route it through the filter, turn on a filter mode too.

SID / voice selection and channel mode

CC	Name	What it does
104	CC_SID1	Narrow this channel to SID 1 only (any nonzero value selects, 0 clears)
105	CC_SID2	Narrow this channel to SID 2 only
106	CC_SID3	Narrow this channel to SID 3 only
107	CC_SID4	Narrow this channel to SID 4 only
108	CC_VCE1	Narrow this channel to voice position 1 on every chip it can use
109	CC_VCE2	Narrow this channel to voice position 2
110	CC_VCE3	Narrow this channel to voice position 3
111	CC_SPLY	Toggle full polyphony for this channel without silencing held notes
119	CC_GTEN	Toggle automatic gate (gate follows note-on/note-off) - on by default
72	CC_VELM	Toggle velocity mode (velocity scales decay) - off by default
126	CC_MONO	Force this channel monophonic, silencing what it currently holds
127	CC_POLY	Force this channel fully polyphonic, silencing what it currently holds

Modulation and timing

One LFO, one arpeggiator and one portamento (glide) engine per channel, all driven by a shared 1kHz tick. These are also the same fields a loaded patch sets (see [Patches](#)) - a live CC change here overwrites whatever the current patch set, and the next Program Change overwrites it back.

CC	Name	What it does
0	CC_LFOW	LFO waveform: triangle, sawtooth, square, or sample & hold
2	CC_LFOR	LFO rate
3	CC_LFOD	LFO depth - 0 turns the LFO off entirely; this is also the tick's cue to skip the channel, so an idle LFO costs nothing
4	CC_LFOT	LFO destination: pitch, pulse width, or filter cutoff
5	CC_PORT	Portamento (glide) time - 0 is off; a freshly triggered note glides in from the previous note this channel played

CC	Name	What it does
6	CC_ARPM	Arpeggiator pattern: up, down, up/down, random, as-played, or table (see Arp tables)
10	CC_ARPR	Arpeggiator rate - free-running unless a MIDI clock is present, in which case it is read as a clock division instead
11	CC_ARPO	Arpeggiator octave range (repeats the held pattern up to 3 extra octaves) - ignored in table mode, see Arp tables
12	CC_ARPE	Arpeggiator on/off for this channel - held notes feed the arpeggiator's input pattern instead of sounding directly while this is on
30	CC_ARPT	Arp table select (0-15) - which table slot `CC_ARPM's "table" pattern reads; only matters once the mode is switched to table, see Arp tables

If your board has PAL/PWM LFO pitch modulation set as the aftertouch target ([AT_VIBRATO](#)), channel pressure becomes a natural vibrato-on-pressure controller without touching [CC_LFOD](#) at all.

Second LFO

A fully independent second oscillator, same shape as the first, stacked on top of it rather than replacing it. If both LFOs are pointed at the same destination their outputs simply add before the one register write - two LFOs both targeting pitch, for example, sum into a single combined vibrato rather than one overwriting the other.

CC	Name	What it does
13	CC_LF02W	LFO 2 waveform: triangle, sawtooth, square, or sample & hold
14	CC_LF02R	LFO 2 rate
15	CC_LF02D	LFO 2 depth - 0 turns LFO 2 off entirely, independent of LFO 1's own depth
16	CC_LF02T	LFO 2 destination: pitch, pulse width, or filter cutoff

Everything that applies to LFO 1 above - the depth-0-costs-nothing behaviour, the patch-vs-live-CC interaction, [AT_VIBRATO](#) - applies identically and independently to LFO 2.

Unison

Stacks 3 real oscillators onto one note instead of one - a thicker, detuned unison sound, MBSID Lead-engine style. This is a genuine tradeoff, not a free thickening: a SID chip only has 3 physical oscillators, so turning unison on for a channel means every note it plays costs all 3 of one chip's voices at once, dropping that chip from 3-voice polyphony down to monophonic for as long as unison stays on. If no SID in the channel's mask has all 3 voices free when a note arrives, the note is simply refused rather than stealing from something else already sounding.

CC	Name	What it does
41	CC_UNIS	Unison mode on/off for this channel (≥ 64 = on, < 64 = off) - only affects the next note-on, does not retroactively thicken anything already sounding
42	CC_UDET	Detune spread (0-127 range but takes the full 0-255 byte) - 0 is no detune at all (3 oscillators in unison), higher values spread the outer two oscillators further apart, up to roughly a semitone at full spread

The center oscillator plays exactly the struck pitch; the other two are detuned up and down by the spread amount around it. Every waveform/envelope/pulse-width CC and patch field still applies to all 3 oscillators together, same as any other multi-voice note - there is currently no way to give the 3 oscillators independently different waveforms or envelopes.

Panic and reset

CC	Name	What it does
120	CC_ASOF	All Sound Off - silences every held voice, on every channel, immediately
121	CC_RACT	Reset All Controllers - same effect as CC_ASOF
123	CC_ANOF	All Notes Off - same effect as CC_ASOF

These three are not channel-scoped - sending any one of them on any channel silences the whole device, SID voices and an FMOpI chip alike. Send this first when starting a new session or a new song, in case a note was left stuck by earlier testing.

Cynthcart

Only relevant on a firmware built with `ONBOARD_CYNTHCART=1` (Cynthcart running through USBSID-Player's own C64 core, see [USBSID-Pico-Cynthcart-manual](#)). These three are fixed CC numbers, not remapped by the general CC map:

CC	Name	What it does
85	CC_CEN	Enable Cynthcart
86	CC_CDI	Disable Cynthcart
87	CC_CRE	Reset Cynthcart

While Cynthcart is active it intercepts and reinterprets a number of the same CC numbers used elsewhere in this document for its own controls (filter, envelope, tremolo, waveform selection, ...) - see the Cynthcart manual for that separate CC list, not this one.

Deprecated

The old "copy/link voice or SID" family (`CC_CVCE/8`, `CC_CSID/9`, `CC_LVCE/40`, `CC_LSID/56`) from before this firmware's per-channel voice templates existed has been removed entirely, not just disabled - there is no field for them left in the CC map at all. CC numbers 8, 9, 40 and 56 are simply unassigned now, the same as any other CC this firmware has no handler for: silently ignored, nothing logged.

Not wired

`CC_BMSB/CC_BLSB` (Bank Select MSB/LSB) and `CC_MOD/CC_MODL` (a second, separate Modulation Wheel MSB/LSB pair distinct from `CC_PWM`) exist as fields in the CC map but have no compiled-in default number (`0xFF/255`, outside the valid 0-127 CC range) and no handler bound to them. Bank Select has nothing to select - with only 32 patches, Program Change alone already reaches every one of them. These four are listed here only for completeness against the full CC map, not because there is anything to send.

Patches

A patch is a saved timbre: waveform, envelope, pulse width, filter settings, LFO, arpeggiator and pitch bend range, all in one shot. Selecting one is a single MIDI Program Change (`0xC0`) - programs 0-31 map to the 32 available patch slots, 32-127 do nothing (there is no Bank Select; with only 32 patches, the program number alone already reaches all of them).

Loading a patch changes the channel's **current** template and pushes it live to every voice already held on that channel - the same thing a live CC change to those same fields would do. It does not touch that channel's routing (voice mask, polyphony, overrides) - a patch is a sound, not a wiring change.

Factory patches

Patch 0 mirrors the compiled-in channel default (plain triangle, full sustain) - selecting it is a safe, silent-to-change no-op. Patches 1-15 are a curated factory set so a fresh board has something to play immediately:

Program	Instrument
1	Sawtooth lead
2	Pulse lead
3	Bass
4	Pluck
5	Organ
6	Filtered lead
7	Slow pad
8	Noise hit

Program	Instrument
9	Reed
10	Sync lead
11	Ring mod lead
12	Vibrato lead
13	Wah pad
14	Wide bass
15	Bright pluck

Patches 16-31 are reserved for your own patches - zeroed (silent) until you save something into them.

Authoring your own patches over SysEx

Patches can be written, read back, saved and reset entirely over MIDI - no config tool required. Every command below is manufacturer ID `0x50: F0 50 <command> ... F7`.

Byte	Command	What it does
0x20	SYSEX_MIDI_PATCH_LOAD	<code>F0 50 20 <slot> <48 nibbles> F7</code> - writes one patch's data into RAM. <code><slot></code> is 0-31.
0x21	SYSEX_MIDI_PATCH_DUMP	<code>F0 50 21 <slot> F7</code> - asks the device to send that patch's current data back out over MIDI (as a <code>0x22</code> message)
0x22	SYSEX_MIDI_PATCH_DATA	Device → host response to a <code>0x21</code> request, and the same 48-nibble shape <code>0x20</code> expects as input
0x2A	SYSEX_MIDI_PATCH_SAVE	<code>F0 50 2A <channel> <slot> F7</code> - the inverse of LOAD: captures the given channel's current live state into a patch slot, RAM only
0x10	SYSEX_MIDI_SAVE	<code>F0 50 10 F7</code> - persists every patch and every channel's current routing to flash
0x11	SYSEX_MIDI_LOAD	<code>F0 50 11 F7</code> - reloads whatever was last saved (also runs automatically at boot and on a MIDI System Reset)
0x12	SYSEX_MIDI_RESET	<code>F0 50 12 F7</code> - erases every saved slot and resets every patch and channel back to compiled defaults

Every field of a patch is sent as two 7-bit-safe nibble bytes (values `0x00-0x0F` each), 48 nibbles per patch in total, in this exact order: waveform+envelope+pulse-width template (5 fields), filter cutoff (a 16-bit field, sent as 4 nibbles), resonance, filter routing, filter mode, LFO 1 waveform/rate/depth/destination (4 fields), arpeggiator mode/rate/octaves (3 fields), pitch bend range, LFO 2 waveform/rate/depth/destination (4 fields), and unison on/off + detune spread (2 fields) - 23 fields, 48 nibbles. The last 6 fields (LFO 2 and unison) were appended after this firmware's first release of patch authoring - an old 36-nibble dump sent to a firmware built with these fields is refused (logged, not misread) rather than silently interpreted wrong; re-export it

from an up to date `patch_editor.py` to pick up the new fields.

`SYSEX_MIDI_PATCH_LOAD` only changes RAM - send `SYSEX_MIDI_SAVE` afterwards if you want it to survive a reboot. Loading a patch into slot 0-15 quietly overwrites one of the factory instruments above in RAM until the next `SYSEX_MIDI_LOAD/reboot/SYSEX_MIDI_RESET`; nothing stops you doing this, it just means slots 0-15 are not truly "read-only," only pre-populated.

The easiest way to actually build patches by hand rather than hand-crafting SysEx bytes is `patch-editor/patch_editor.py` in this repository - a small graphical tool with a knob/slider/switch for every field above, Send/Read buttons that speak this exact protocol, and an importer for real `.s3i/.op2/.wopl` instrument files (see that tool's own help text for FMOpI banks). It also has a raw-hex box for pasting a SysEx line copied from anywhere else, including `patch-editor/midibox_sid_presets.md` - roughly 500 MIDIBOX SID V2 presets already converted into this firmware's patch format, ready to paste and try.

Arp tables

A GoatTracker/SidWizard-style arp/chord table: instead of picking one of the arpeggiator's 5 fixed shapes (up, down, up/down, random, as-played), a table is a small authored sequence of relative semitone offsets from the arpeggiator's held root note (the first key you pressed) - your own custom arpeggio shape, not just an algorithmic one. Everything about how the arpeggiator itself runs - free-running or MIDI-clock-synced timing, `CC_ARPR`'s rate/division, `'CC_ARPE` turning it on and off - works identically whether it is stepping one of the 5 fixed shapes or a table; only **which offset comes next** is different.

There are 16 table slots, each holding up to 16 steps. Table 0 is a repeating major triad (0, +4, +7 semitones) as a safe, non-silent default; tables 1-15 start empty (0 steps, meaning an arpeggiator switched to table mode with an empty table plays nothing) until you author one.

Using a table

Three separate things have to line up for a table to actually play:

1. `CC_ARPT` picks **which** table slot (0-15) to read.
2. `CC_ARPM` set to "table" (the value at the very top of its range, e.g. 127) picks table mode **as the pattern shape** - selecting a table with `CC_ARPT` alone does not switch a channel into table mode by itself.
3. `CC_ARPE` turns the arpeggiator on for the channel, same as any other arp mode.

`CC_ARPO` (octave range) is ignored in table mode - a table's steps are already fully authored by you; layering an automatic octave repeat on top would just be a second, conflicting way of extending the same pattern.

Authoring tables over SysEx

Same manufacturer ID and LOAD/DUMP/DATA shape as patches and FMOpI patches, its own commands:

Byte	Command	What it does
0x27	SYSEX_MIDI_ARPTABLE_LOAD	F0 50 27 <slot> <36 nibbles> F7 - writes one table into RAM. <slot> is 0-15.
0x28	SYSEX_MIDI_ARPTABLE_DUMP	F0 50 28 <slot> F7 - requests that table's current data back
0x29	SYSEX_MIDI_ARPTABLE_DATA	Device → host response to a 0x28 request, and what 0x27 expects as input

18 fields per table, each sent as two nibbles same as everything else in this document: 16 step offsets (signed, -128 to +127 semitones, sent as the raw byte's two nibbles) followed by loop start and step count - 36 nibbles total. `step_count` of 0 means the table is empty/unauthored and the arpeggiator produces silence if switched to it; `loop_start` is which step the pattern jumps back to once it reaches the end, so a table can have a distinct intro before settling into a repeating tail (e.g. steps 0-2 play once, then it loops steps 1-2 forever with `loop_start = 1`, `step_count = 3`).

`patch-editor/patch_editor.py`'s "Arp Table" tab covers authoring these visually - a spinbox per step, loop-start/step-count sliders, and Send/Read buttons, plus a one-click "Select on channel" action that sends `'CC_ARPT` and switches `CC_ARPM` to table mode for you (still send `CC_ARPE` separately to actually turn the arpeggiator on).

Tables are saved/loaded/reset by the same `SYSEX_MIDI_SAVE/LOAD/RESET` that covers patches and channel routing - see [Saving your settings](#).

Saving your settings

`SYSEX_MIDI_SAVE/SYSEX_MIDI_LOAD/SYSEX_MIDI_RESET` (see above) persist every channel's routing, every patch, every arp table, and the CC map itself, all together, to a dedicated flash region separate from the board's own general configuration. A same-layout firmware reflash does not wipe this; a firmware upgrade that changes the size of the saved data (a new field added to a future version) falls back to compiled defaults safely rather than loading garbage.

A MIDI System Reset (0xFF, a real-time status byte, not a SysEx message) reloads whatever was last saved - the expected behaviour for a synth's "reset," restoring your last saved sound rather than blanking to bare factory defaults. `SYSEX_MIDI_RESET` is the separate, deliberate factory-reset command if that is what you actually want.

Channel volume (`CC_VOL`) is the one exception: it is never saved or restored by any of the three commands above, on purpose - there is no single "correct" volume to persist, and a client (a MIDI player, ASID, your own sequencer) is expected to set it explicitly rather than inherit whatever a previous session happened to leave it at. Expect a System Reset or a fresh boot to come back at a flat, audible default volume regardless of what you last saved, even though every patch, routing and arp table setting comes back exactly as saved.

FMOpI

Some USBID-Pico configurations have an FMOpI-capable chip installed in place of (or alongside) a

SID - either real OPL2 silicon, or a clone chip's software emulation of one (for example SIDKick-pico's onboard FMOpI emulation). Where it exists, it is one specific chip slot on your board, entirely separate from the SID voice pool: 2-operator FM synthesis, 9 simultaneous notes, its own small set of MIDI controls. Everything in this section only applies if your board actually has one configured - check with your board's config tool if you are not sure.

Targeting a channel at FMOpI

By default every MIDI channel targets a SID. A channel opts in to FMOpI explicitly, with **CC_FMEN** (CC 88): send **127** to target that channel at the FMOpI chip instead, **0** to send it back to targeting a SID. This is independent of the channel's own SID voice mask/routing - it does not matter what SID(s) a channel is otherwise configured for, **CC_FMEN** overrides that entirely for as long as it is on.

A channel must have **CC_FMEN enabled before Program Change or notes on it reach the FMOpI chip at all.** Sending notes to an FMOpI-targeted channel without ever setting **CC_FMEN** first does nothing; sending notes to a channel that used to target FMOpI but was reset back to SID with **CC_FMEN 0** plays through the SID pool as normal, not the OPL chip.

CC_FMEN is sticky, per-channel, firmware state - it does not reset itself between sessions or songs. If you use both SID and FMOpI channels across different pieces, explicitly reset each channel to whichever it should be at the start of every song rather than assuming the board's current state; see [patch-editor/strudel-demos/](#) for worked examples of exactly this.

FMOpI patches

Program Change on an FMOpI-targeted channel selects one of 32 FMOpI patch slots, the same Program-Change-selects-a-slot idea as the SID side, but a completely different, much smaller patch shape: 2 FM operators (a modulator and a carrier), each with its own multiplier, key-scale level/total-level, attack/decay, sustain/release and waveform, plus a shared feedback/algorithm setting (classic FM/series, or additive/parallel).

Slots 0-5 are compiled-in factory instruments:

Program	Instrument
0	Organ
1	Electric piano
2	Bass
3	Brass
4	Bell
5	Pad

These were hand-designed from FM synthesis theory rather than copied from any particular commercial sound bank - a reasonable starting point, not a claim of authenticity to any specific ROM. Slots 6-31 start silent, open for your own patches.

Authoring works the same way as the SID side, mirrored one-for-one, still manufacturer **0x50**:

Byte	Command	What it does
0x23	SYSEX_FMOPL_PATCH_LOAD	F0 50 23 <slot> <22 nibbles> F7 - writes one FMOpl patch into RAM
0x24	SYSEX_FMOPL_PATCH_DUMP	F0 50 24 <slot> F7 - requests that patch's data back
0x25	SYSEX_FMOPL_PATCH_DATA	Device → host response to a 0x24 request, and what 0x23 expects as input
0x2B	SYSEX_FMOPL_PATCH_SAVE	F0 50 2B <channel> <slot> F7 - captures the given channel's current live FMOpl state into a patch slot, RAM only

11 raw bytes per patch (2 operators x 5 fields, plus one shared feedback/algorithm byte), 22 nibbles total. These patches are not yet included in the general SYSEX_MIDI_SAVE/LOAD/RESET flash persistence - they live in RAM only for now and reset to the factory 6 on every reboot. `patch-editor/patch_editor.py`'s FMOpl tab covers authoring these the same way it covers the SID side, including an importer for real `.s3i` (Scream Tracker 3), `.op2` (DMX GENMIDI, the format Doom/Heretic/Hexen use) and `.wopl` (OPL3BankEditor/libADLMIDI) instrument files.

Live control

Three controls reach an FMOpl-targeted channel live, on top of notes and Program Change:

CC	Name	What it does
7	CC_VOL	Channel volume - live, rewrites every currently held note's loudness immediately
1	CC_PWM	Modulation wheel - OPL2 has no pulse width to control, so this drives a live modulator-only brightness/FM-intensity boost instead: raising it makes the modulator louder relative to the carrier for a brighter, more overtone-rich tone, without touching overall volume
-	Pitch Bend (0xE0)	Live, same semantics as the SID side - bends every held note on the channel using that channel's bend range

Every other CC that has meaning on a SID-targeted channel (waveform bits, ADSR, filter, LFO, arpeggiator, ...) currently does nothing on an FMOpl-targeted channel - there is no OPL equivalent wired up for those yet.

Known limitations

- No LFO, no arpeggiator, no portamento on an FMOpl-targeted channel yet - notes, Program Change, volume, mod wheel and pitch bend only.
- Velocity is accepted but not yet used.
- A Program Change while a note is still held never retroactively changes that note's own timbre - matches how the SID side already behaves.

- FMOpl patches are RAM-only for now, not yet part of flash save/load/reset.

A note on tuning

The clock rate used to convert a MIDI note into the chip's own pitch registers has, so far, proven harder to pin down than expected - it depends on the exact chip/emulation your board's FMOpl slot actually uses, and the compiled-in default has not been confirmed correct on every configuration tested. If notes sound obviously out of tune, [patch-editor/patch_editor.py](#)'s FMOpl tab has a live, no-flash-needed clock override with a held test note, specifically for dialing this in by ear - see that tool's own help text. Under the hood this is 'SYSEX_FMOPL_SET_CLOCK (0x26): F0 50 26 <6 nibbles> F7, a 3-byte (24-bit) big-endian clock value; an all-zero payload resets to the compiled-in default.

Development

More information about USBSID-Pico is available on the [github repo](#)

License

Software License ~ GNUv2

All code written by me in this repository is licensed under the terms of the GNU General Public License as published by the Free Software Foundation, version 2. Any code in this repository that is not written by me automatically falls under the licensing conditions by the authors of said code as mentioned in the source code header. For more information about this license please read the LICENSE document in the root of the USBSID-Pico repository.

Configtool Software License ~ EPLv2

This program and the accompanying materials are made available under the terms of the Eclipse Public License 2.0 which is available at [eclipse.org](#).

SPDX-License-Identifier: EPL-2.0

USBSID-Pico v1.5 PCB Hardware License ~ CC BY-NC-ND 4.0

USBSID-Pico v1.5 PCB © 2026 by [LouD](#) is licensed under [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International](#)



USBSID-Pico v1.3 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.3 PCB © 2025-2026 by [LouD](#) is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Pico v1.0 PCB Hardware License ~ CC BY-SA 4.0

USBSID-Pico v1.0 PCB © 2024-2026 by [LouD](#) is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



USBSID-Volume-daughterboard v1.0 PCB Hardware License ~ CC BY-SA 4.0

Volume-daughterboard v1.0 PCB © 2026 by [LouD](#) is licensed under
Creative Commons Attribution-ShareAlike 4.0 International



Author: LouD - generated on 2026-09-02